

ЛЕКЦИЯ 13



ВЫЧИСЛИТЕЛЬНЫЕ СИСТЕМЫ. ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ ЧАСТЬ 2

- Примеры вычислительных систем SIMD
- Примеры вычислительных систем MIMD
- Вычислительные системы с нетрадиционным управлением
- Перспективные типы процессоров



Вычислительные системы класса SIMD

- Векторные вычислительные системы
- Матричные вычислительные системы
- Ассоциативные вычислительные системы
- Систолические вычислительные системы

Вычислительные системы класса MIMD

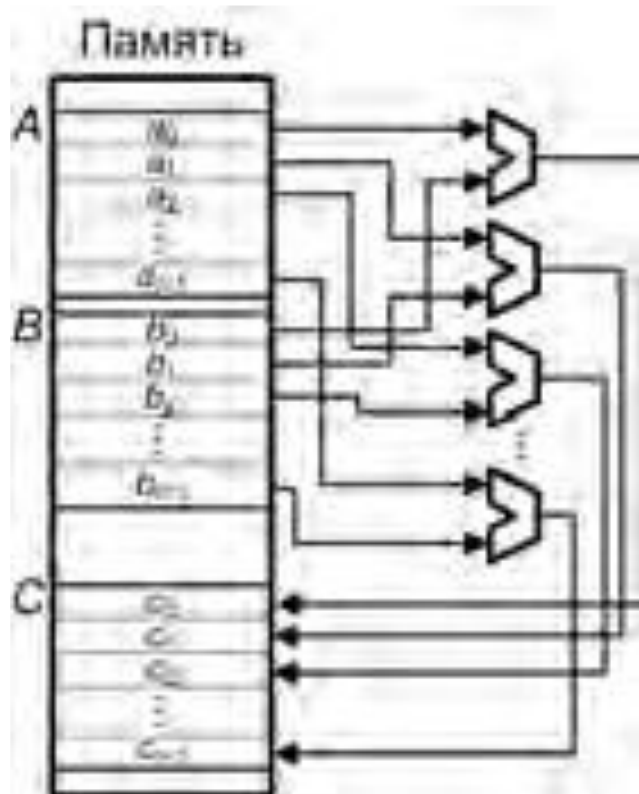
- Параллельные векторные системы (PVP)
- Массивно-параллельные системы (MPP)
- Кластерные системы

ВС с нетрадиционным управлением

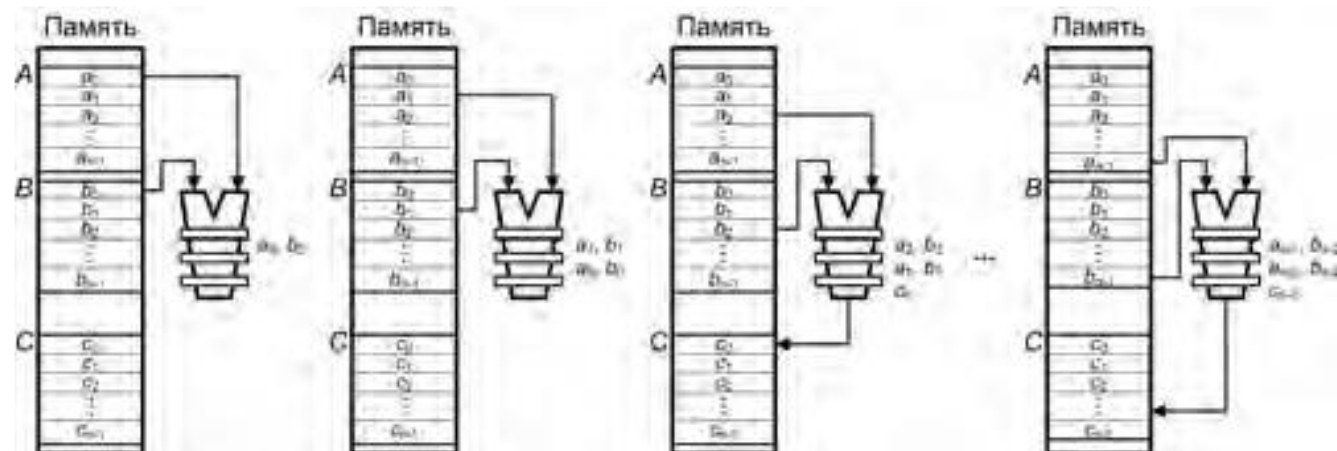
- ВС с управлением от потока данных
- ВС с управлением по запросу

Векторный процессор – процессор, в котором операндами некоторых команд могут выступать массивы данных – векторы

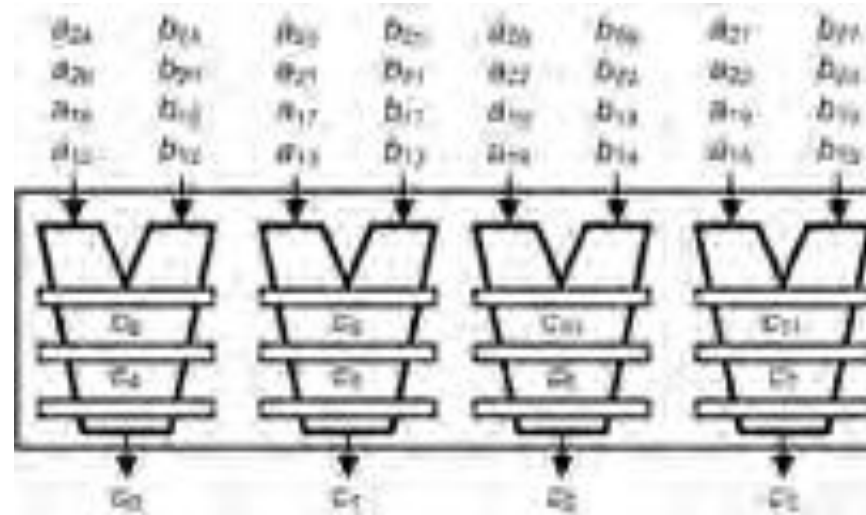
Векторно-параллельная обработка

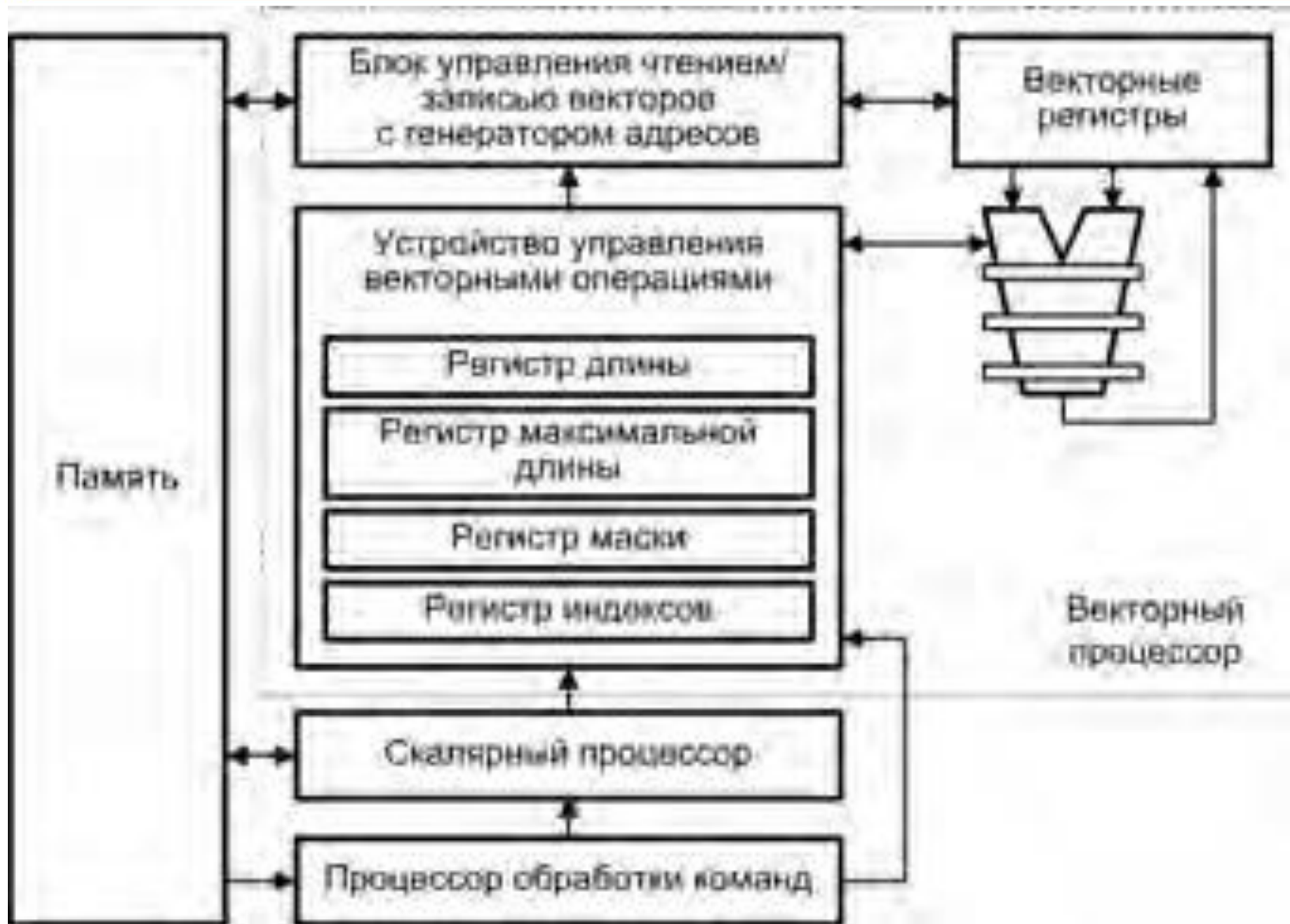


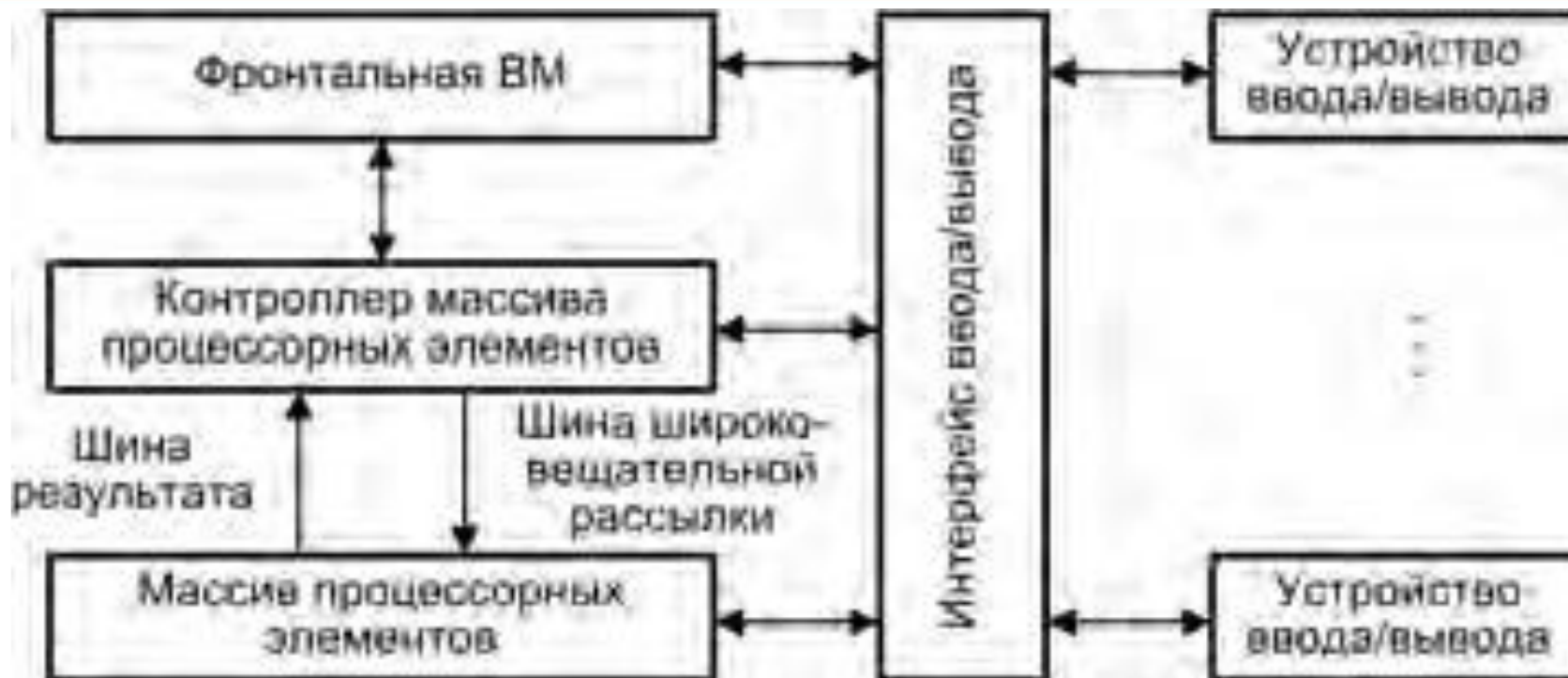
Векторно-конвейерная обработка



Параллельная обработка векторов несколькими конвейерными ФБ



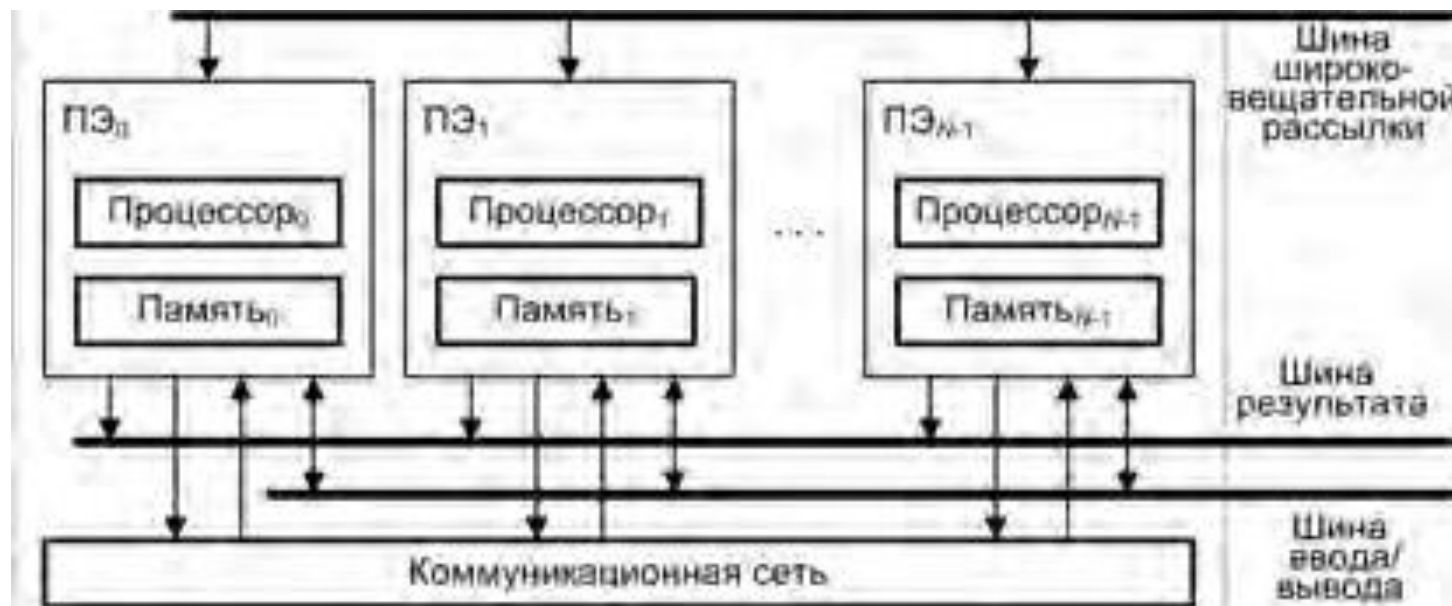




- *Контроллер массива процессорных элементов (КМП)* выполняет последовательный программный код, реализует операции переходов, транслирует в МПЭ команды, данные и сигналы управления по *шине широковещательной рассылки*
- Результаты обработки данных в массиве процессоров транслируются в КМП по *шине результата*
- *Фронтальная VM* – универсальная VM, на которую дополнительно возлагается задача загрузки программ и данных в КМП



Архитектура типа
«процессорный элемент-
процессорный элемент»



Архитектура типа
«процессор-память»



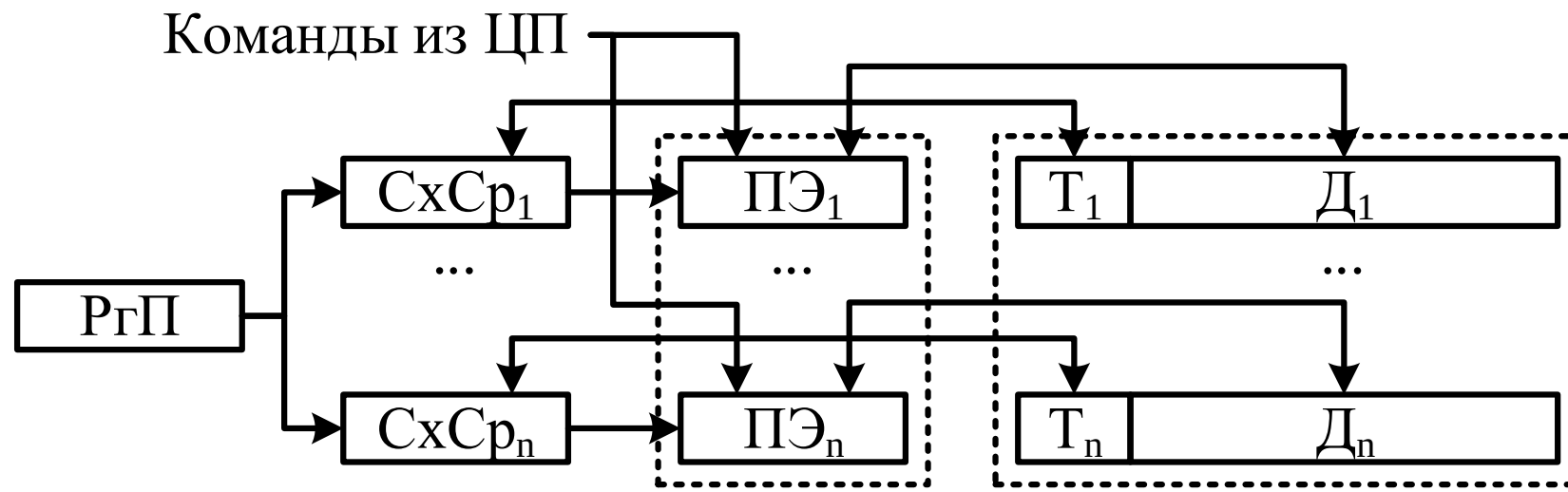
Состав ПЭ:

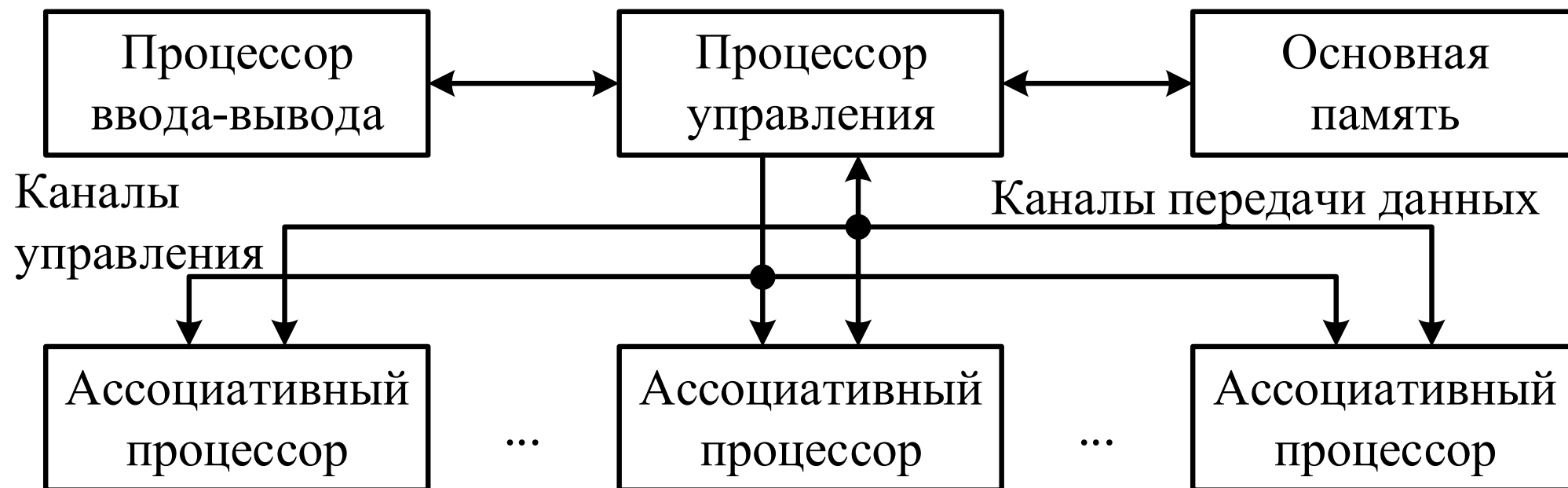
- Арифметико-логическое устройство
- Регистры данных
- Сетевой интерфейс
- Номер ПЭ
- Регистр флага разрешения маскирования (F)
- Локальная память





- Ассоциативный процессор – специализированный процессор, реализованный на базе ассоциативного запоминающего устройства (АЗУ), где доступ к информации осуществляется не по адресу операнда, а по отличительным признакам, содержащимся в самом операнде
- От АЗУ традиционного применения ассоциативный процессор (АП) отличают две особенности:
 - ❑ Наличие средств обработки данных
 - ❑ Возможность параллельной записи во все ячейки, для которых было зафиксировано совпадение с ассоциативным признаком (*мультизапись*)

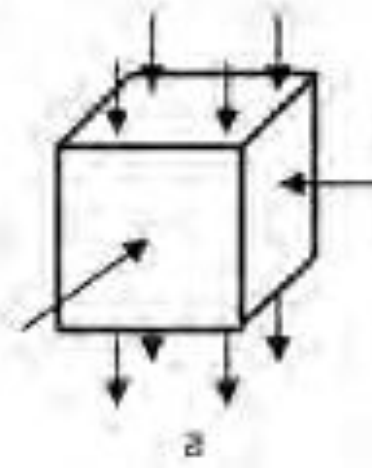
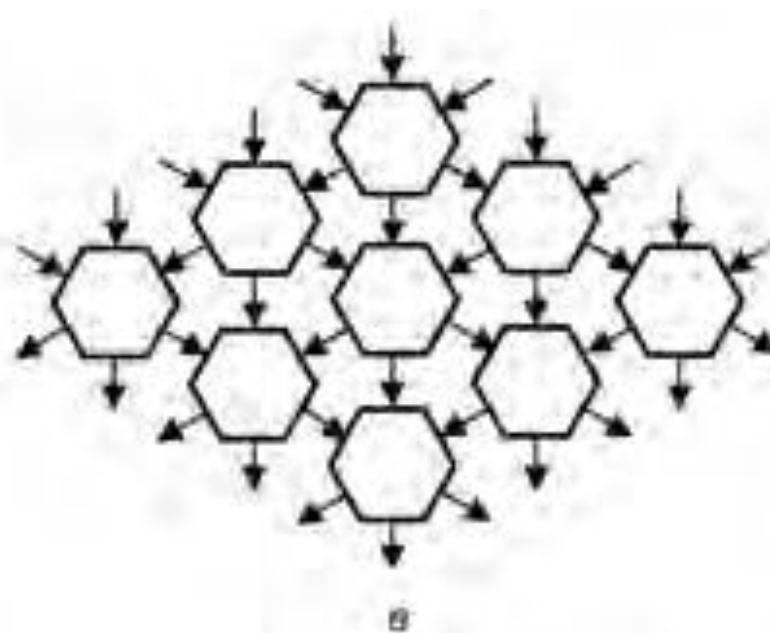
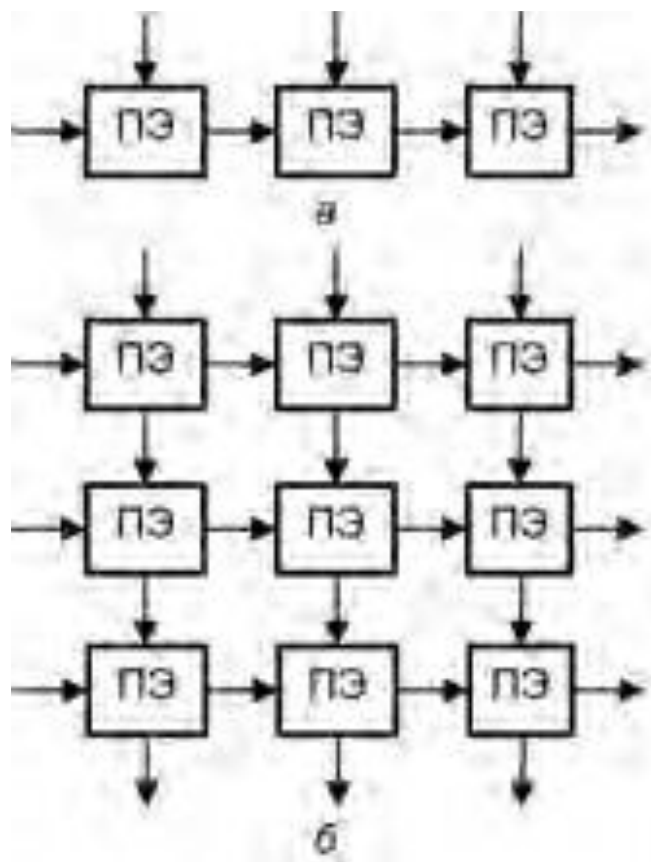






Систолическая структура – однородная вычислительная среда из ПЭ, совмещающая в себе свойства конвейерной и матричной обработки:

- Вычислительный процесс – непрерывная и регулярная передача данных от одного ПЭ к другому без запоминания промежуточных результатов вычисления
- Каждый элемент входных данных выбирается из памяти однократно и используется столько раз, сколько необходимо по алгоритму
- Образующие систолическую структуру ПЭ однотипны и каждый из них может быть менее универсальным, чем процессоры обычных многопроцессорных систем
- Потоки данных и управляющих сигналов обладают регулярностью, что позволяет объединять ПЭ локальными связями минимальной длины
- Алгоритмы функционирования позволяют совместить параллелизм с конвейерной обработкой данных



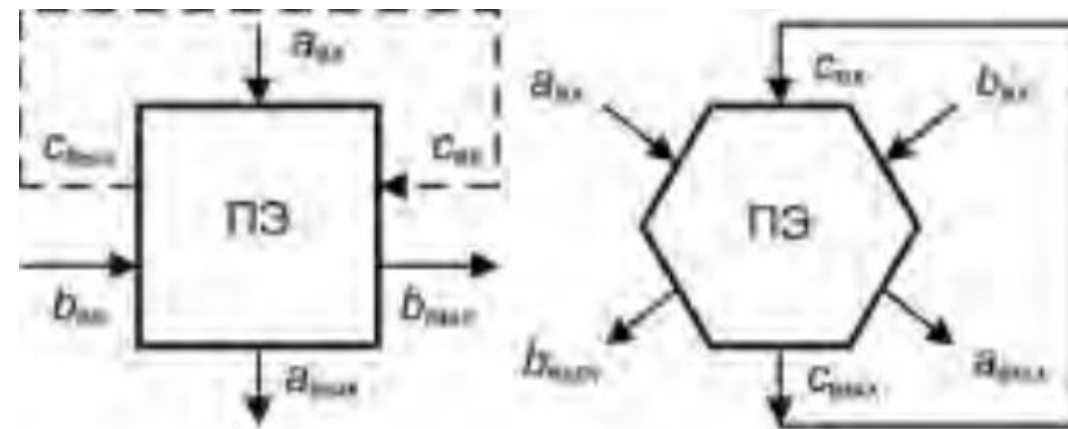
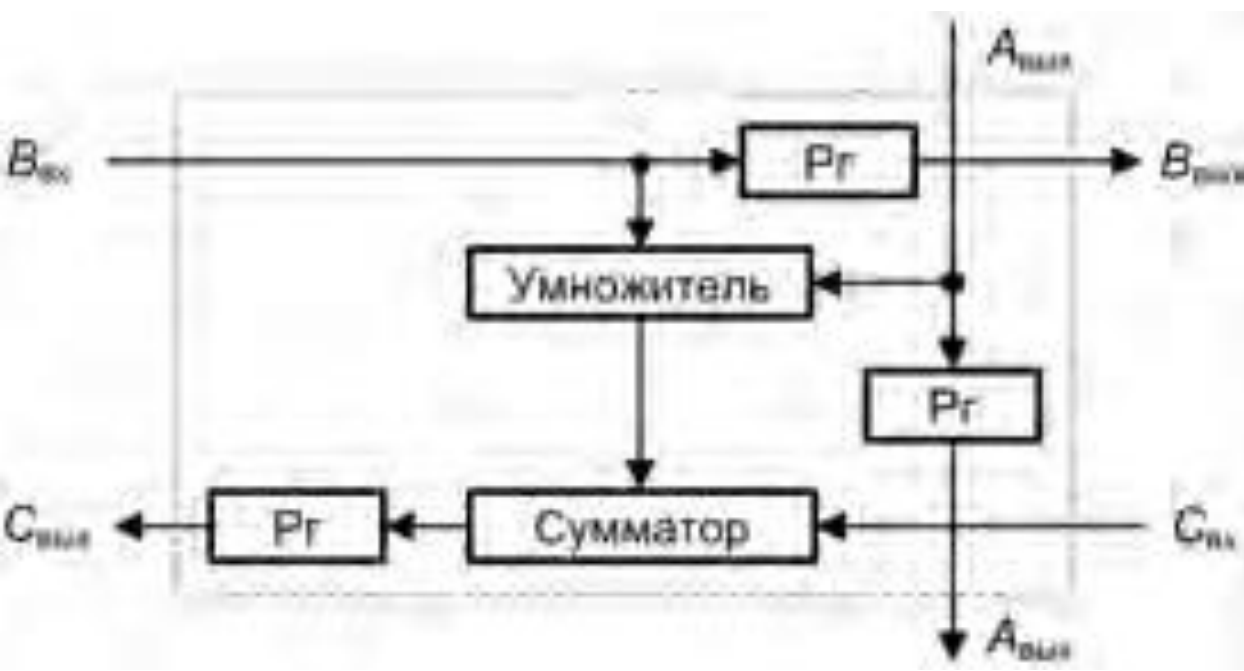
Конфигурация систолических матриц:

- **а** – линейная
- **б** – прямоугольная
- **в** – гексагональная
- **г** – трёхмерная

Пример систолической структуры

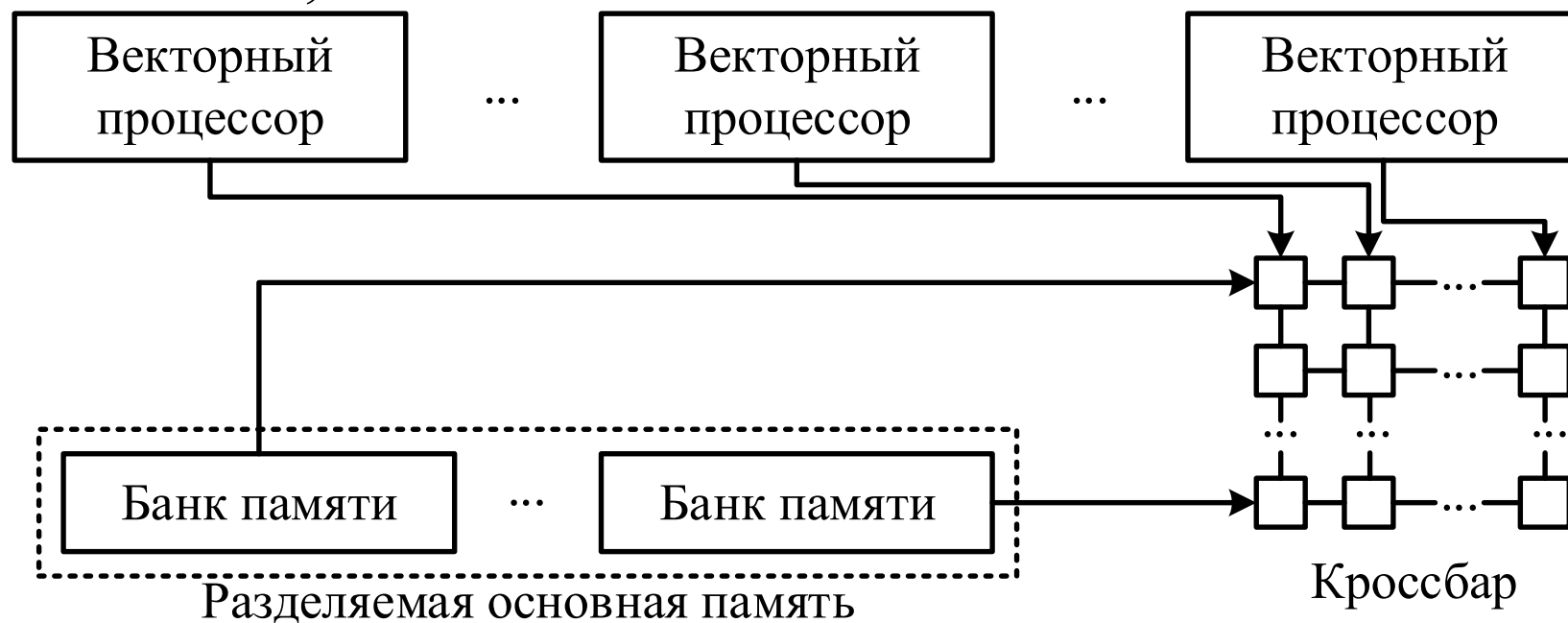
- На вход ПЭ подаются два операнда $a_{\text{ВХ}}$, $b_{\text{ВХ}}$, а выходят операнды $a_{\text{ВЫХ}}$, $b_{\text{ВЫХ}}$ и частичная сумма $c_{\text{ВЫХ}}$
- На n -м шаге работы систолической системы ПЭ выполняет операцию:

$$c_{\text{ВЫХ}}^{(n)} = c_{\text{ВХ}}^{(n-1)} + a_{\text{ВХ}}^{(n-1)} \times b_{\text{ВХ}}^{(n-1)}, \text{ при этом } a_{\text{ВЫХ}}^{(n)} = a_{\text{ВХ}}^{(n-1)} \text{ и } b_{\text{ВЫХ}}^{(n)} = b_{\text{ВХ}}^{(n-1)}$$





- Параллельные векторные системы (PVP, Parallel Vector Processor) – SMP-системы, где роль процессорных элементов исполняют векторно-конвейерные процессоры
- PVP-система содержит небольшое число индивидуальных векторных процессоров, связанных широкополосным коммутатором типа «кроссбар»
- Количество ПЭ в системе может достигать 512, типовые значения – 8–16





Каждый модуль системы (узел) содержит:

- ☐ Процессор
- ☐ Локальный банк оперативной памяти
- ☐ Два коммуникационных процессора (маршрутизатора): для передачи команд и данных
- ☐ Жёсткие диски и/или другие устройства ввода-вывода

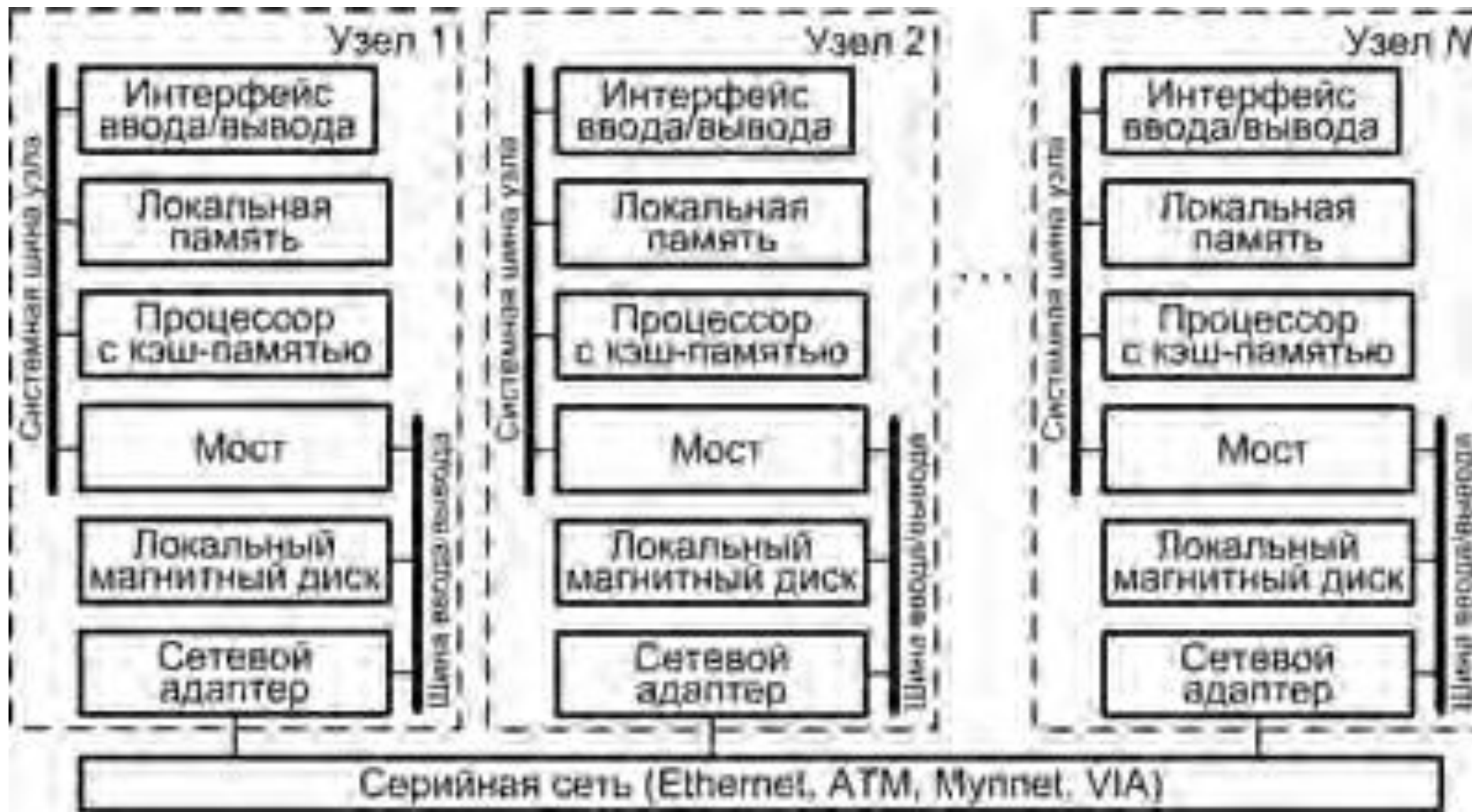
Хост-компьютер – ядро системы (планировщик заданий)



- Вычислительный узел может работать автономно (стандартные микропроцессоры, локальная память, подсистема ввода/вывода)
- Каждый узел содержит сетевой адаптер, используемый для объединения узлов
- Модель распределённой памяти – доступ к памяти других узлов обеспечивается путём асинхронного обмена сообщениями
- Сеть соединений обычно проектируется под конкретную систему для обеспечения высокой пропускной способности и малых задержек
- Система хорошо масштабируется (до тысяч узлов)
- Работа системы координируется главной ВМ (хост-компьютером) или одним из узлов, выполняющим роль главной ВМ
- На каждом узле установлена ОС или её урезанный вариант, если имеется хост-компьютер с полноценной ОС
- Вычисления представляют собой множество процессов, имеющих отдельные адресные пространства

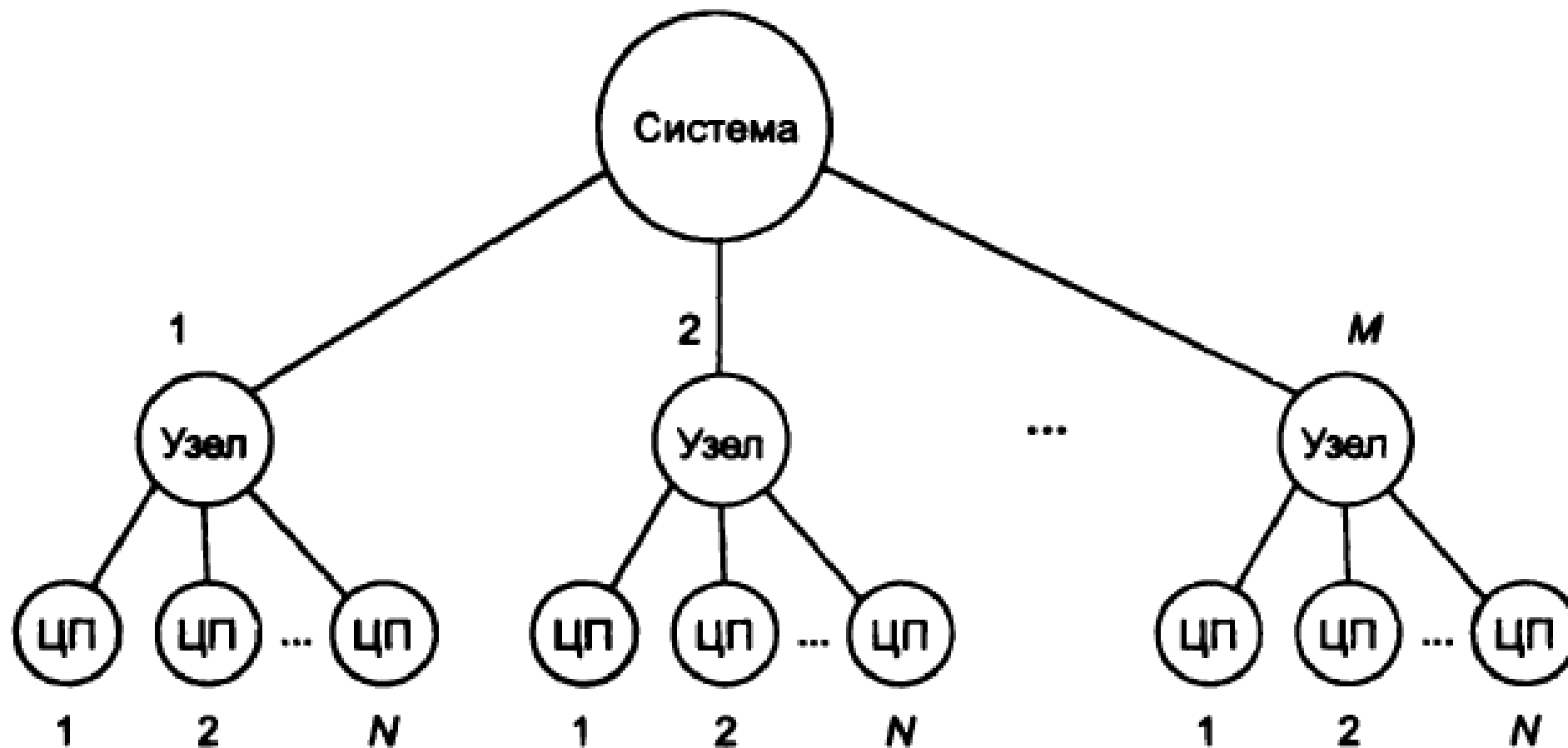


Кластер – группа взаимно соединённых узлов, работающих совместно и составляющих единый вычислительный ресурс, создавая иллюзию единственной ВМ





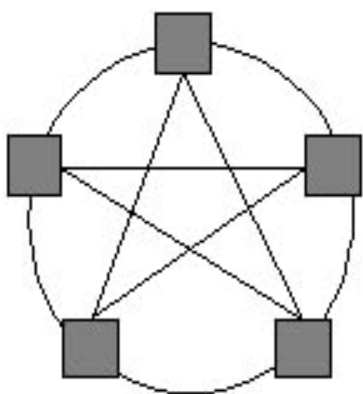
- **Абсолютная масштабируемость.** Кластер в состоянии содержать десятки узлов, каждый из которых представляет собой мультипроцессор
- **Наращиваемая масштабируемость.** Кластер строится так, что его можно наращивать, добавляя новые узлы небольшими порциями
- **Высокий коэффициент готовности.** Каждый узел кластера – самостоятельная ВМ или ВС, отказ одного из узлов не приводит к потере работоспособности кластера
- **Низкое соотношение цена/производительность.** Кластер любой производительности можно создать, соединяя стандартные «строительные блоки», при этом его стоимость будет ниже, чем у одиночной ВМ с эквивалентной вычислительной мощностью



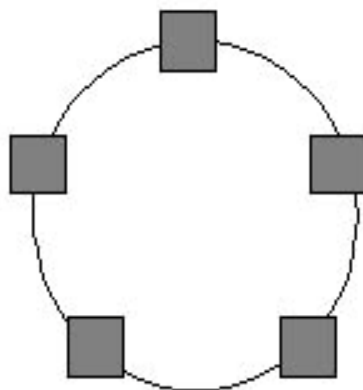
Если $N < M$ – кластерная конфигурация, Если $N > M$ – созвездие



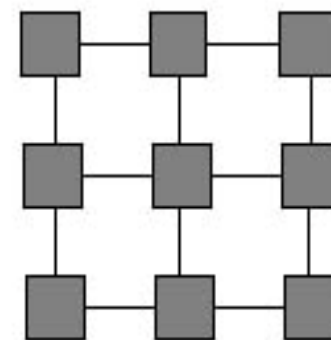
- В мультикомпьютерах для взаимодействия, синхронизации и взаимоисключения параллельно выполняемых процессов используется передача данных между процессорами вычислительной среды
- **Топология сети передачи данных** – структура линий коммутации между процессорами вычислительной системы
- Для построения кластерной системы во многих случаях используют **коммутатор (switch)**, через который процессоры кластера соединяются между собой.
- В любой момент времени каждый процессор может принимать участие только в **одной** операции приёма-передачи данных



Полный граф
(*completely-
connected graph
or clique*)



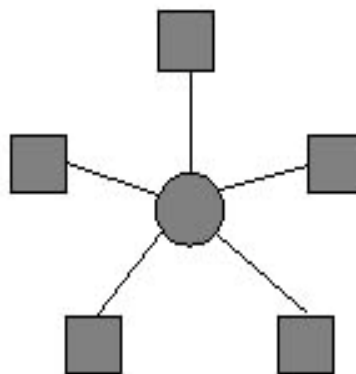
Кольцо
(*ring*)



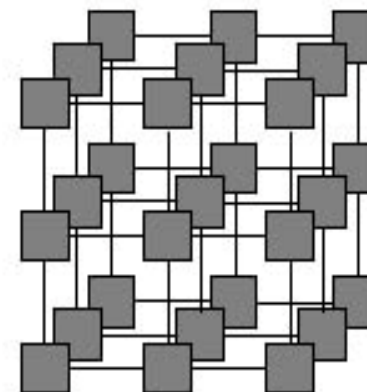
Решетка
(*mesh*)



Линейка
(*linear array
or farm*)



Звезда
(*star*)



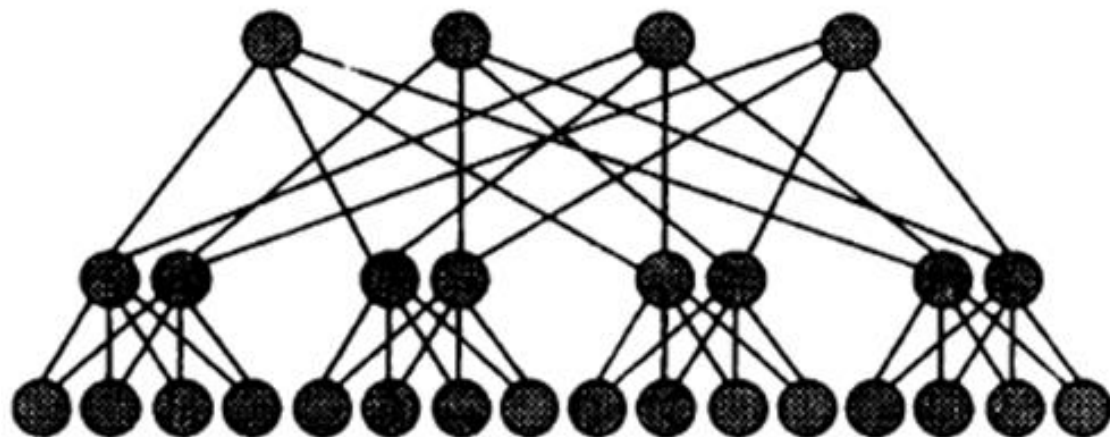
Гиперкуб
(*hypercube*)



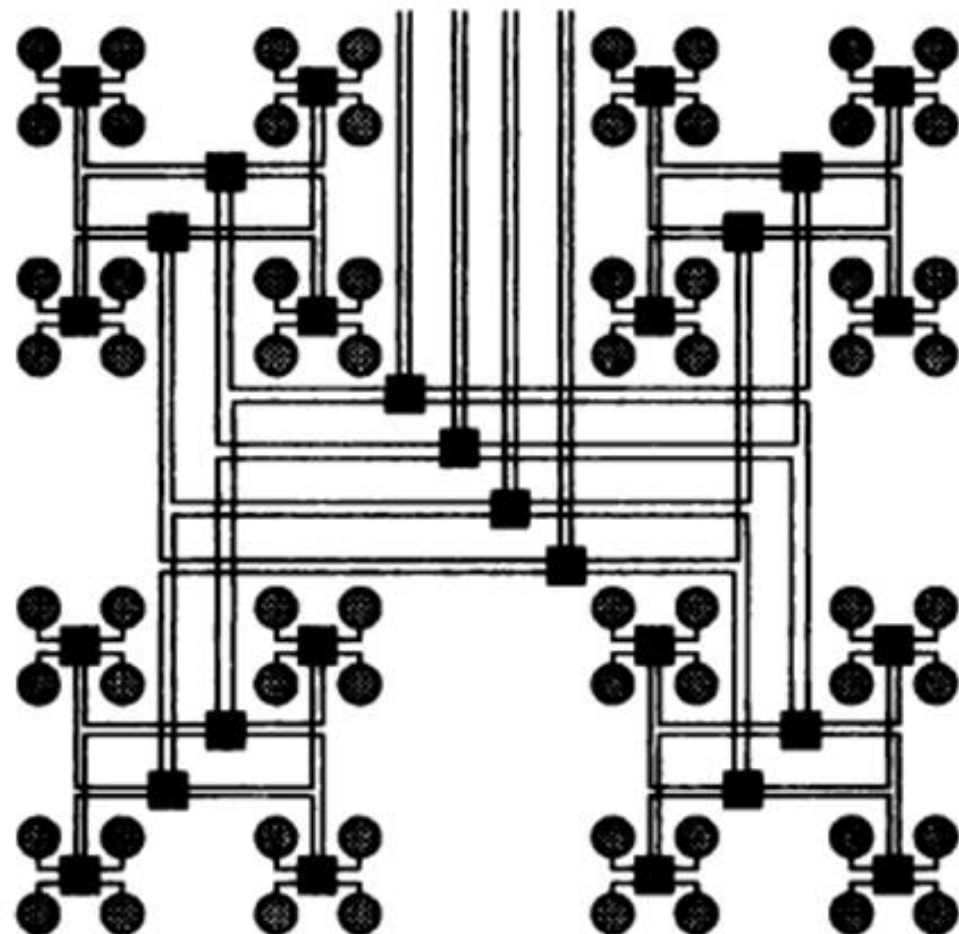
- **Диаметр** – максимальное расстояние между двумя процессорами сети; характеризует максимально-необходимое время для передачи данных между процессорами
- **Связность** (*connectivity*) – показатель, характеризующий наличие разных маршрутов передачи данных между процессорами сети (например, минимальное количество дуг, которое надо удалить для разделения сети передачи данных на две несвязные области)
- **Ширина бинарного деления** (*bisection width*) – минимальное количество дуг, которое надо удалить для разделения сети передачи данных на две несвязные области **одинакового размера**
- **Стоимость** – общее количество линий передачи данных в многопроцессорной вычислительной системе



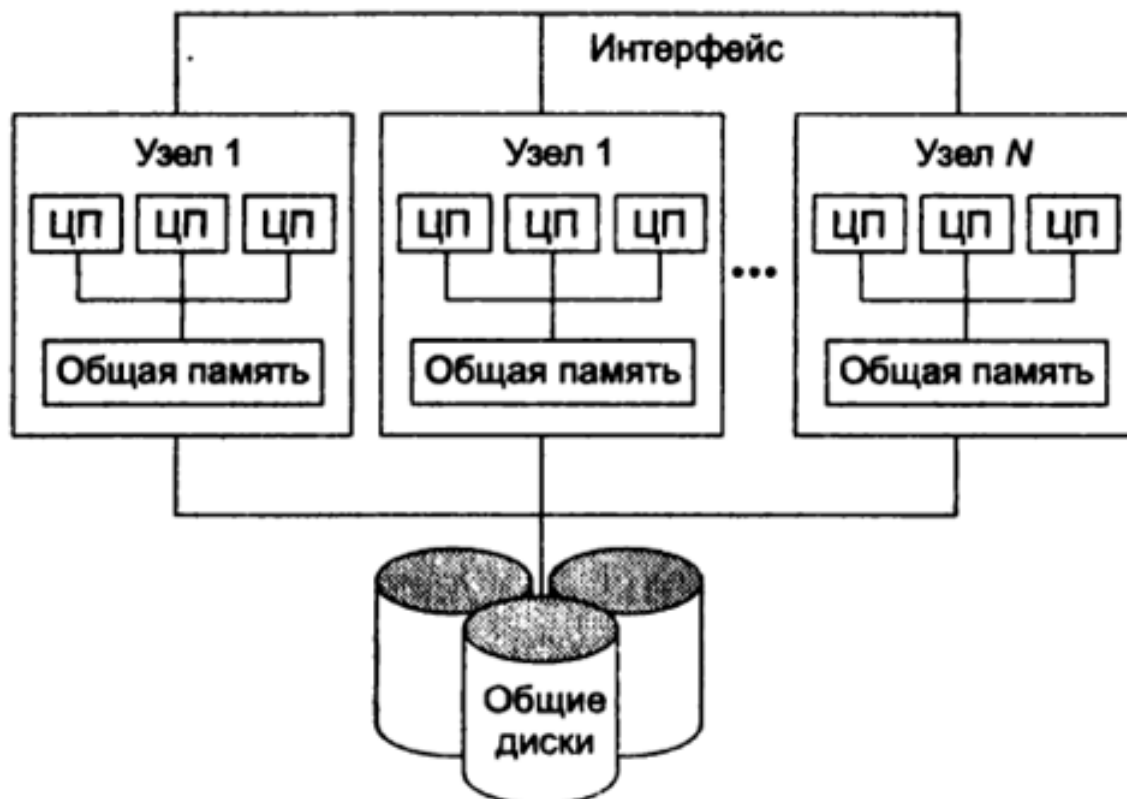
Топология	Диаметр	Ширина бисекции	Связность	Стоимость
Полный граф	1	$p^2/4$	$(p-1)$	$p(p-1)/2$
Звезда	2	1	1	$(p-1)$
Линейка	$p-1$	1	1	$(p-1)$
Кольцо	$2\lfloor\sqrt{p}/2\rfloor$	2	2	p
Гиперкуб	$\log_2 p$	$p/2$	$\log_2 p$	$p \log_2 p/2$
Решётка ($N=2$)	$\lfloor p/2 \rfloor$	$2\sqrt{p}$	4	$2p$



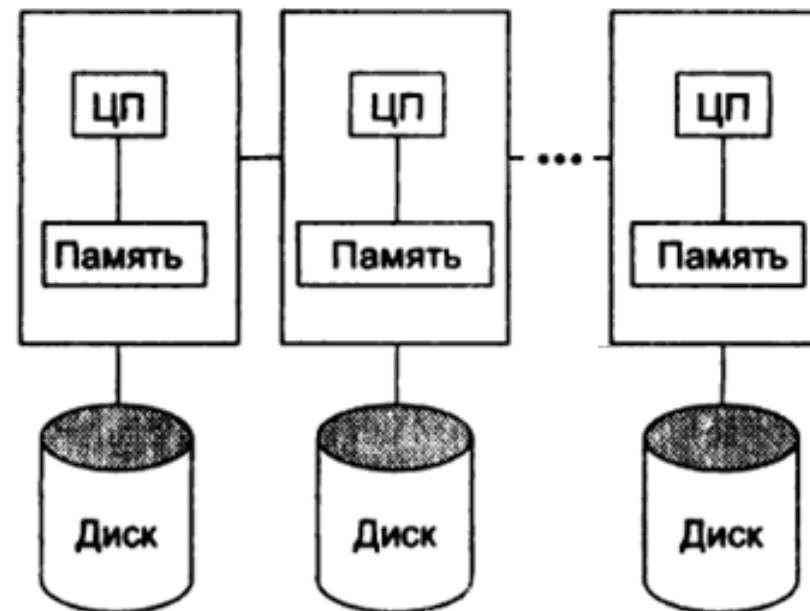
Вид «сбоку»



Вид «сверху»



Однородный (UDMA)



Неоднородный (NUDMA)

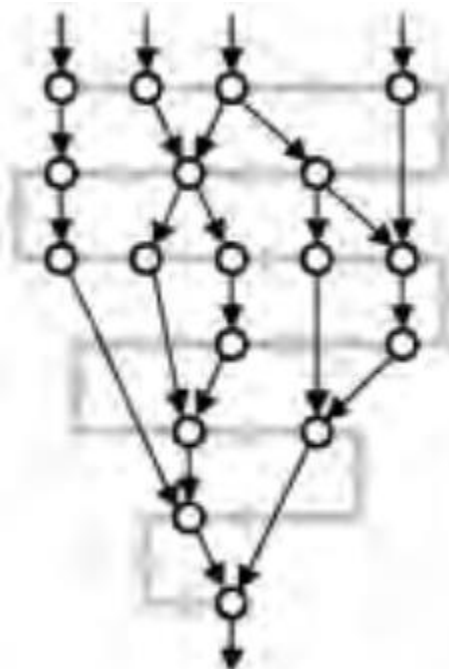


- **Надёжность** – работа системы без сбоев в определённых условиях в течение определённого времени. Единица измерения – среднее время наработки на отказ (*Mean Time Between Failure, MTBF*)
- **Отказоустойчивость** – способность вычислительной системы продолжать действия, заданные программой, после возникновения неисправностей
 - ☐ Избыточность
 - ☐ Автоматическая реконфигурация
 - ☐ Кластерные системы

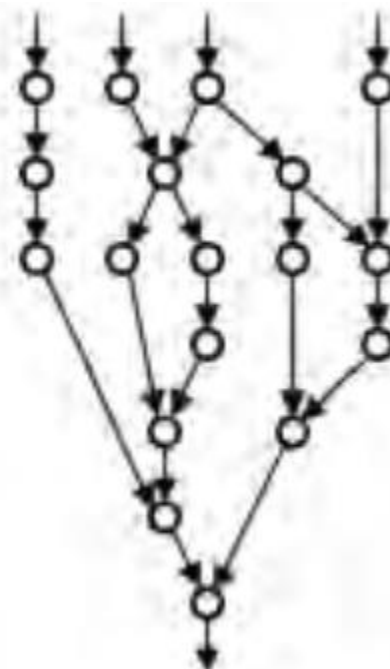


Механизмы управления последовательностью вычислений:

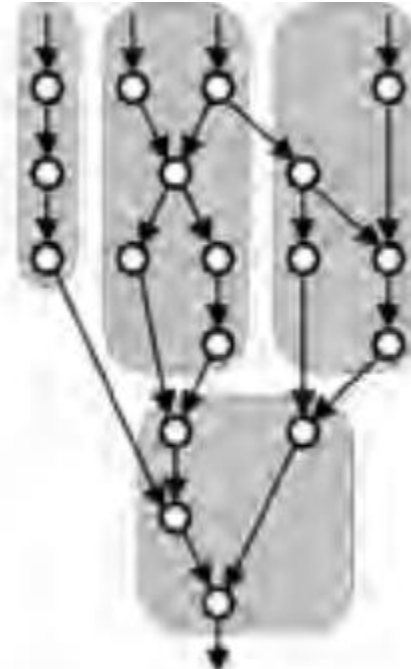
- Команда выполняется, после того как выполнена предшествующая ей команда последовательности (фон-неймановский механизм)
- Команда выполняется, когда становятся доступными её операнды (*управляемый данными* (data driven) или *поточковый* (dataflow))
- Команда выполняется, когда другим командам требуется результат её выполнения (*управления по запросу* (demand driven) или *редукционным*)



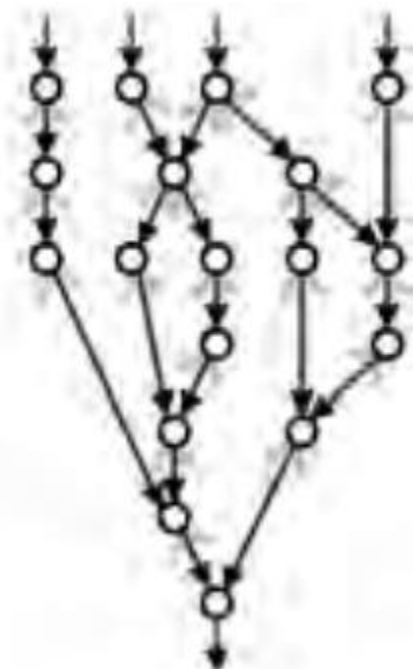
фон-неймановская



поточковая



мультипоточковая

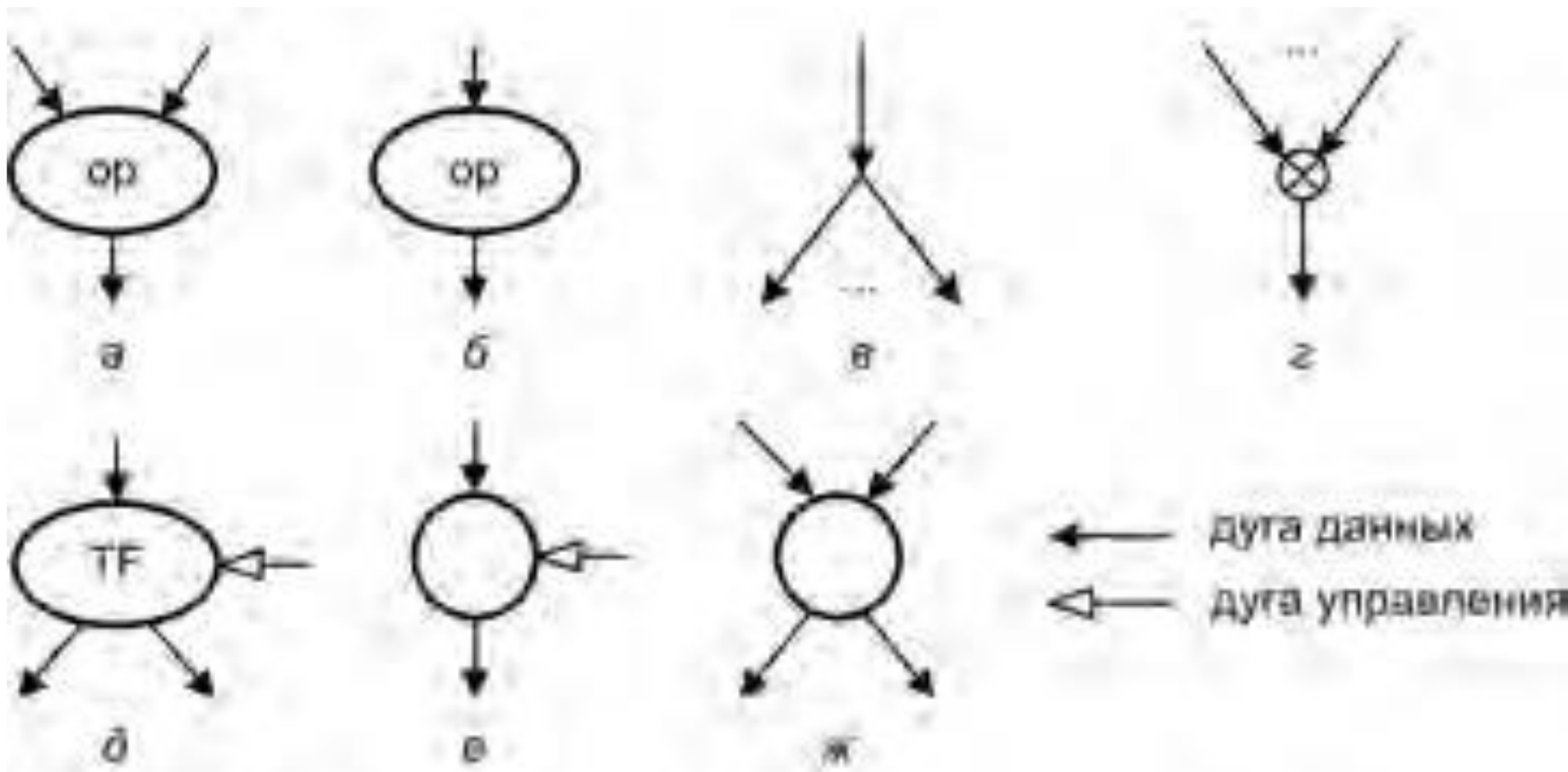


редукционная

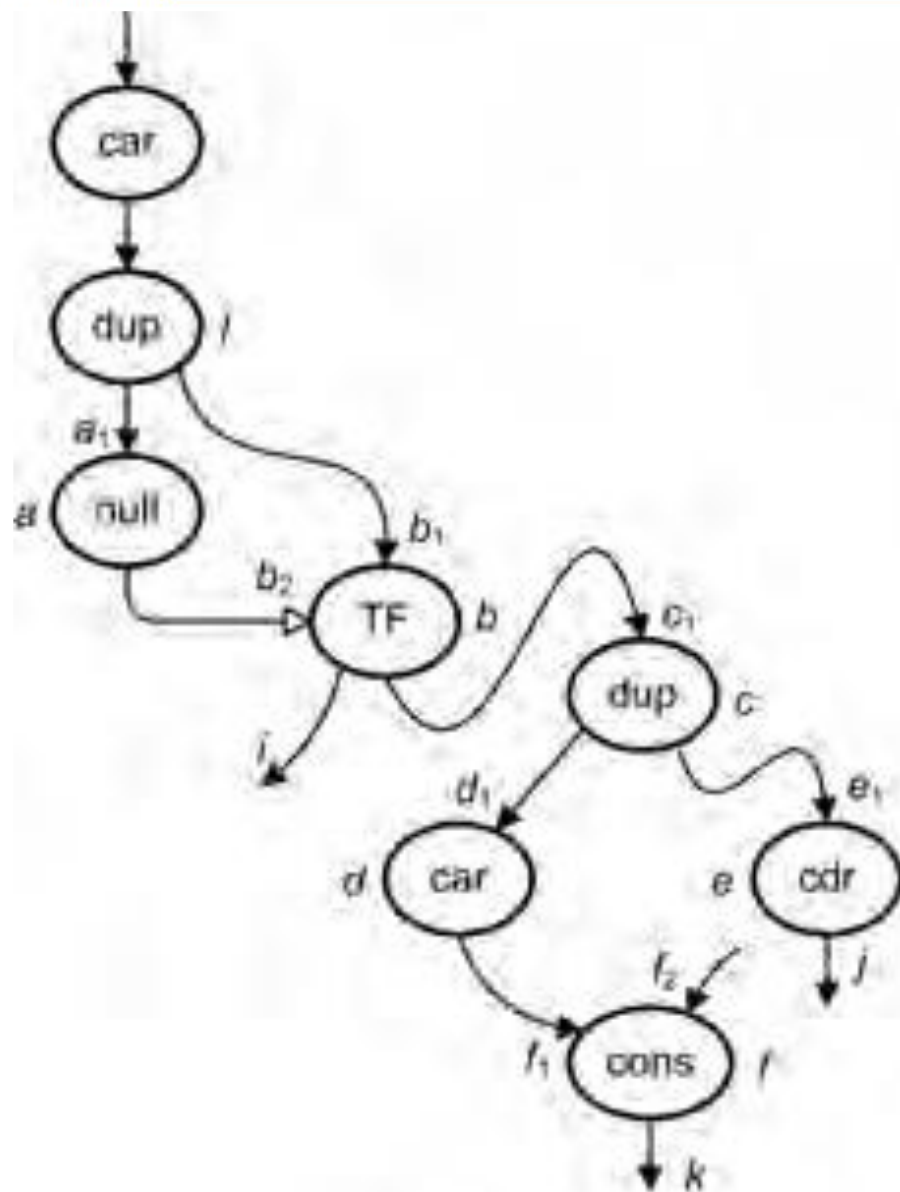
- В потоковой ВС используется ориентированный граф, называемый *графом потоков данных* (dataflow graph)
- Процесс потоковой обработки может работать в конвейерном режиме: после обработки первого набора входных сигналов на вход графа может быть подан второй
- Промежуточные результаты (*токены*) первого вычисления не обрабатываются совместно с промежуточными результатами второго и последующих вычислений
- Все известные потоковые вычислительные системы могут быть отнесены к двум основным типам: *статическим* и *динамическим*
- Динамические потоковые ВС реализуются по схемам:
 - ❑ *Архитектура с помеченными токенами*
 - ❑ *Архитектура с явно адресуемыми токенами*



- *двухвходовая операционная вершина* – операции производятся над данными, поступающими с левой и правой входных дуг, а результат выводится через выходную дугу
- *одновходовая операционная вершина* – операции выполняются над входными данными, результат выводится через выходную дугу
- *вершина ветвления* осуществляет копирование входных данных и их вывод через две выходных дуги
- *вершина слияния* – данные поступают только с какого-нибудь одного из двух входов и без изменения подаются на выход
- *вершина управления* – существует в следующих вариантах:
 - ❑ *TF-коммутатор* – если значение на управляющем входе истинно, то входные данные выводятся через левый выход, при ложном – через правый
 - ❑ *вентиль* – при истинном значении на входе управления данные выводятся через выходную дугу
 - ❑ *арбитр* – данные, поступившие на один из двух входов первыми, следуют через левую дугу, а прибывшие впоследствии — через правую выходную дугу



а — двухвходовая операционная вершина;
 б — одновходовая операционная вершина;
 в — вершина ветвления;
 г — вершина слияния;
 д — TF-коммутатор; е — вентиль; ж — арбитр

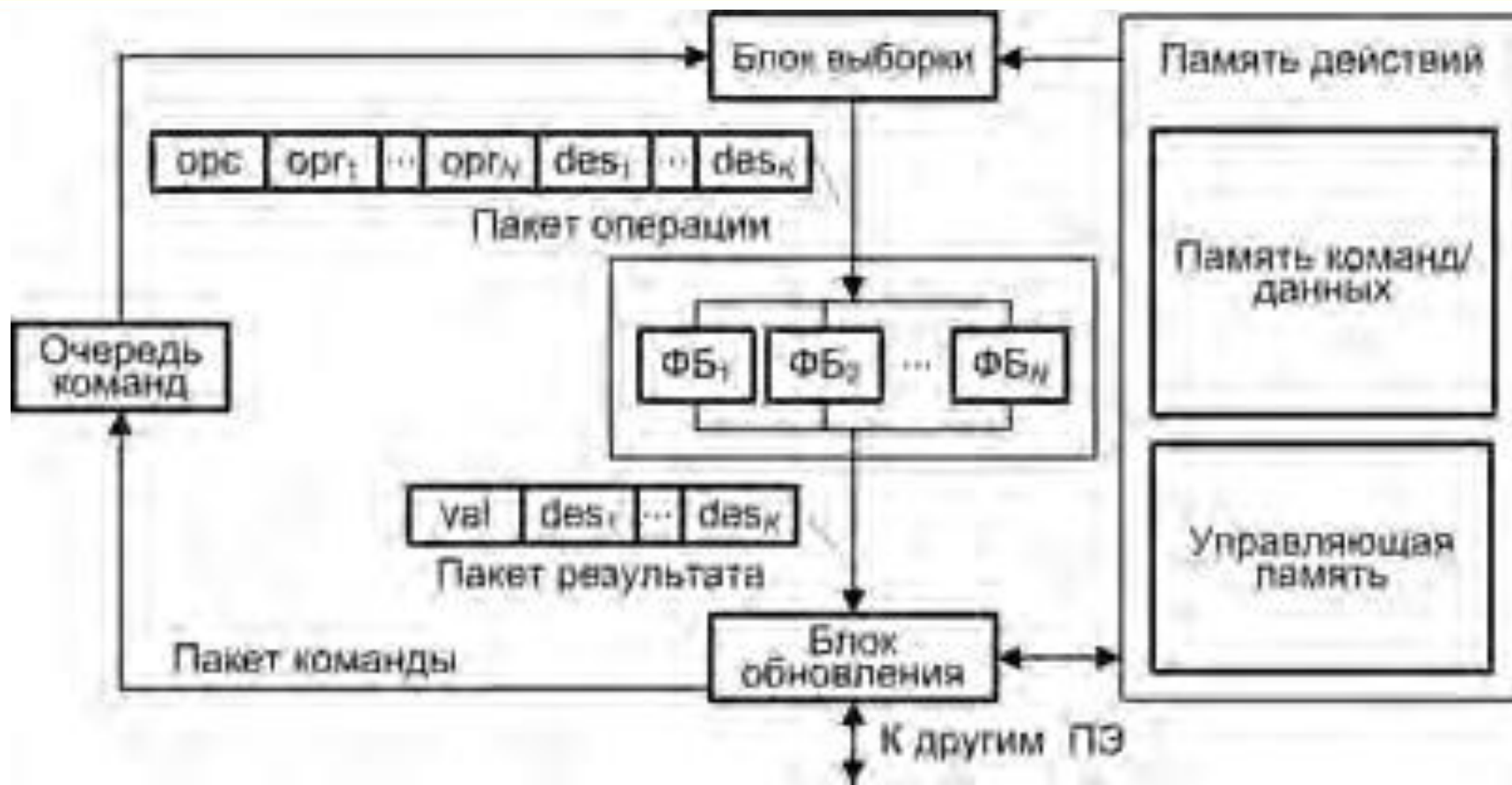


а

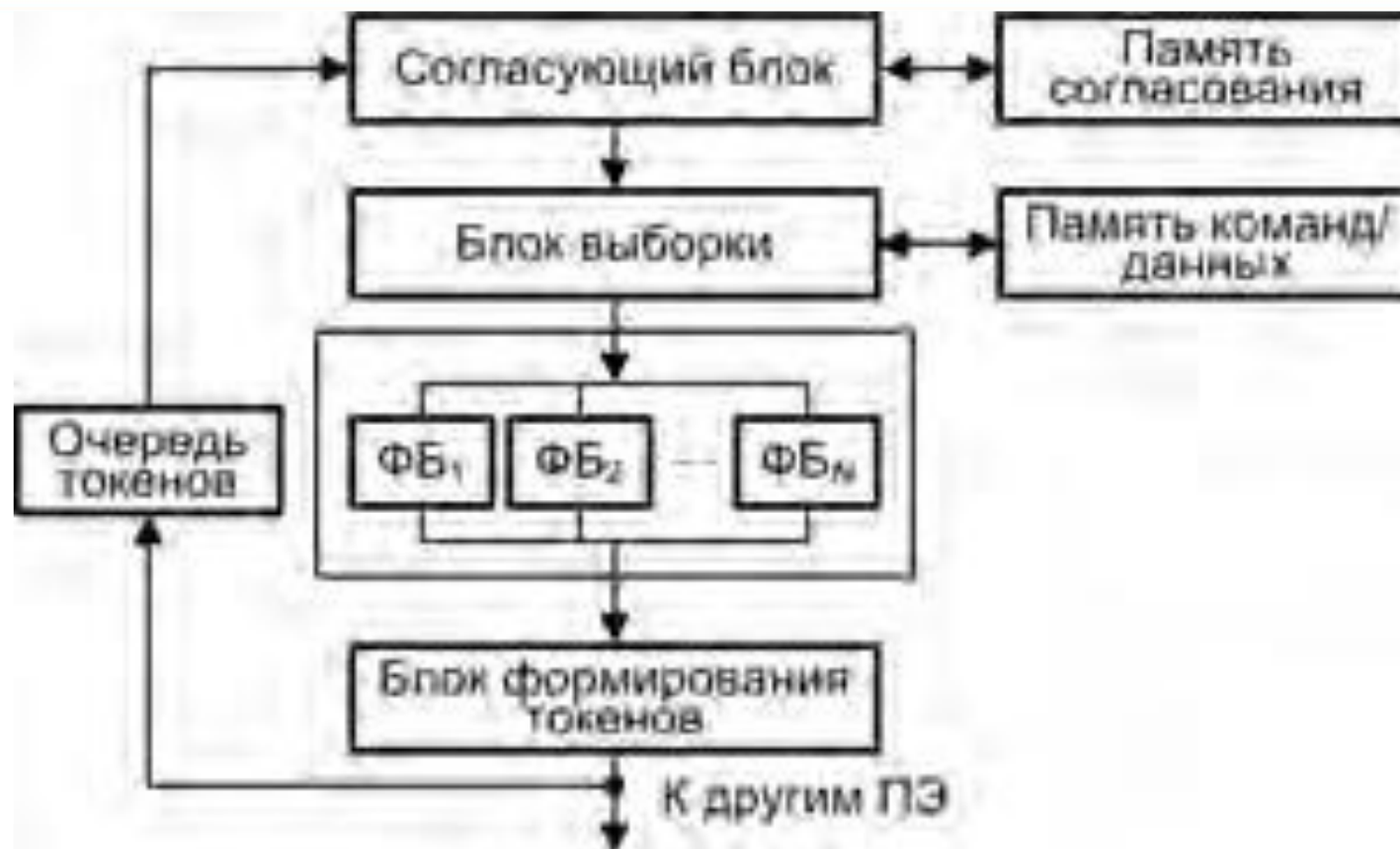
opc – код операции;
opr1, opr2 – поля операндов;
des – адрес назначения;
pres – биты наличия данных

	opc	opr1	opr2	des	pres
	car			l	0, 1
l	dup			a ₁ , b ₁	0, 1
a	null			b ₂	0, 1
b	TF			i, c ₁	0, 0
c	dup			d ₁ , e ₁	0, 1
d	car			f ₁	0, 1
e	cdr			j	0, 1
f	cons			k	0, 0

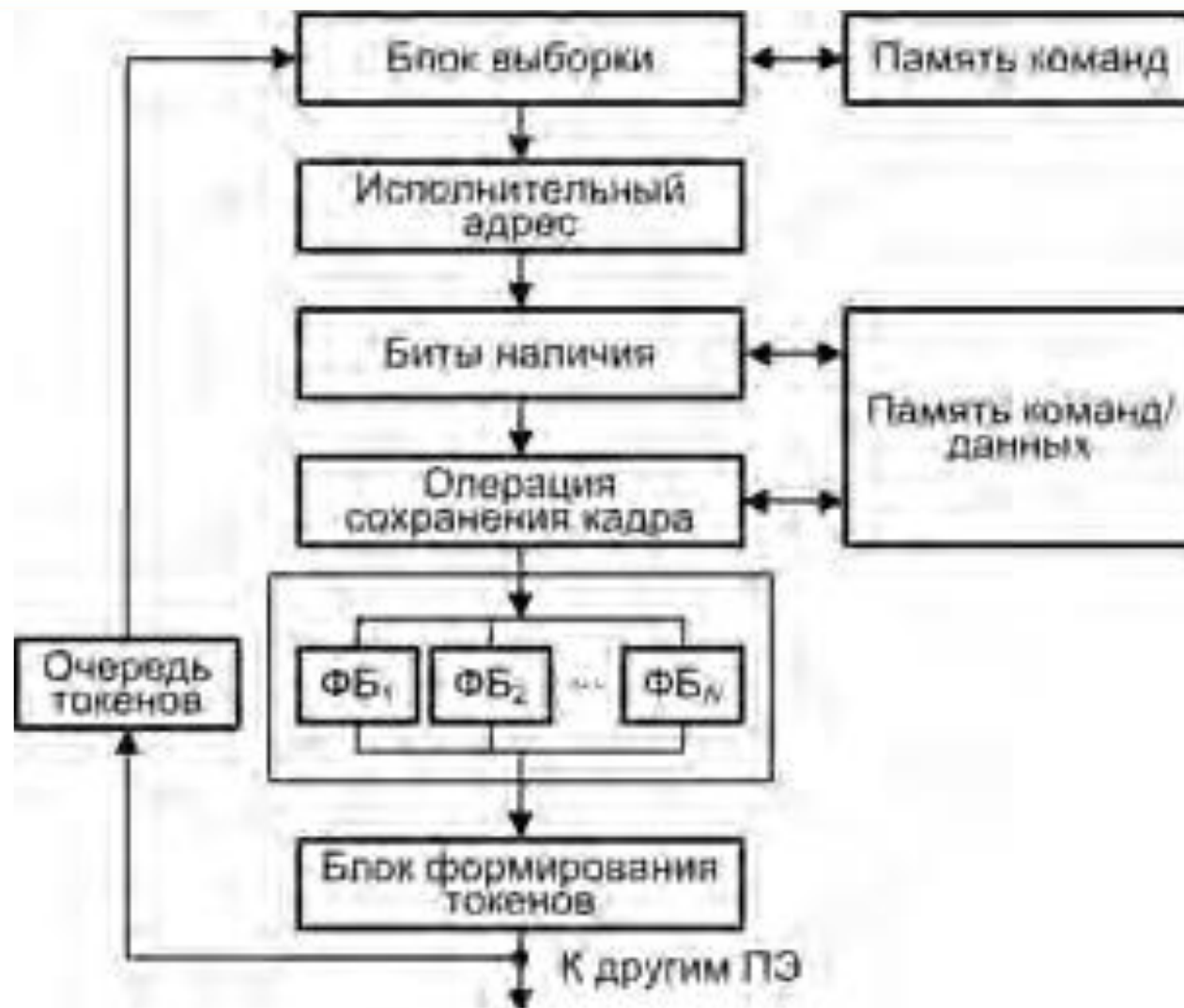
б



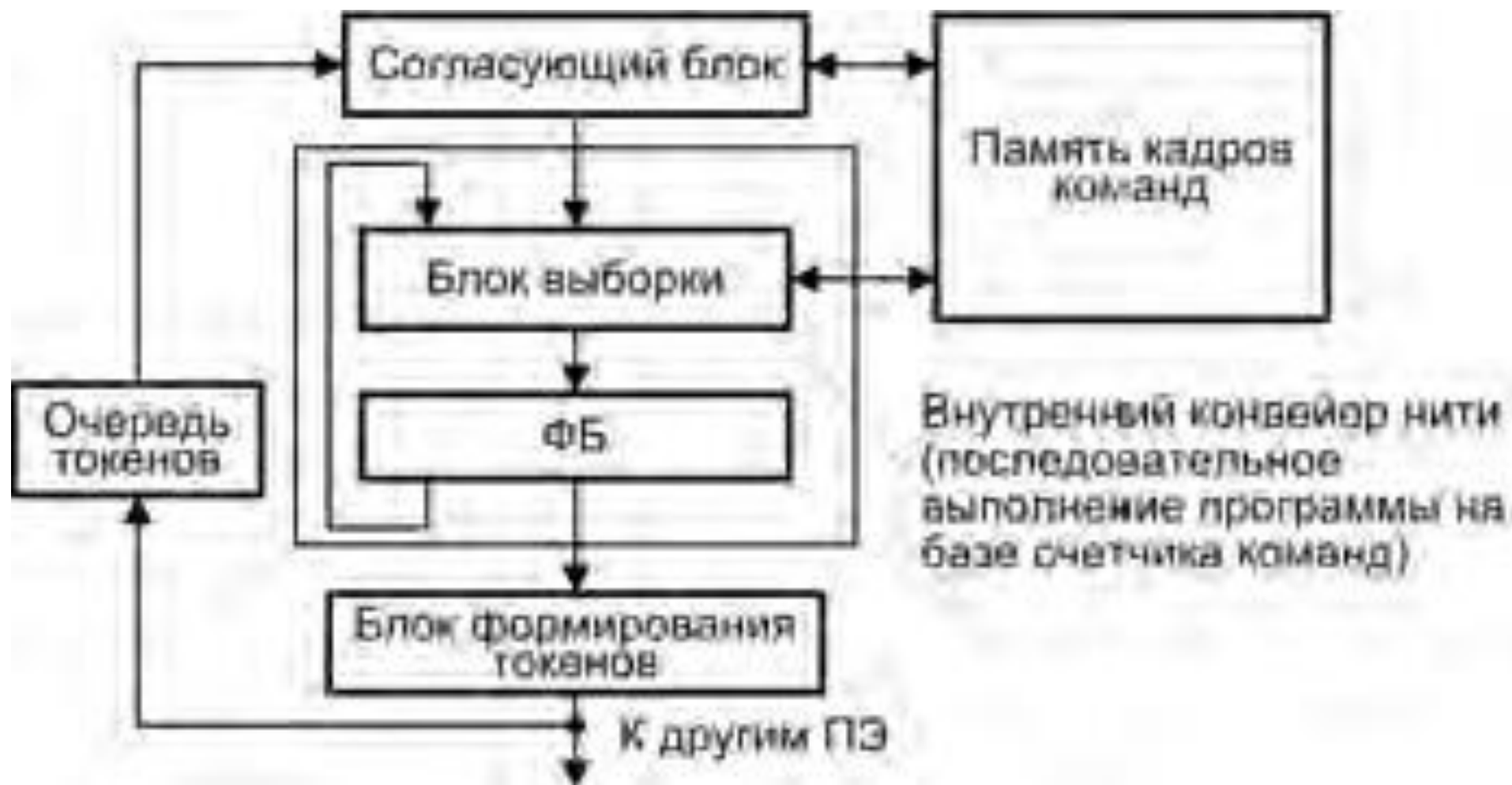
- Вершина активируется, когда на всех её входных дугах присутствует по токену и ни на одном из её выходов токенов нет
- Токен: $\langle v, \langle f, n \rangle, a \rangle$, где v – данные, переносимые токеном, f – текущая функция, n – номер целевой вершины, куда должен поступить данный токен, a – номер дуги, по которой токен должен быть направлен к целевой вершине

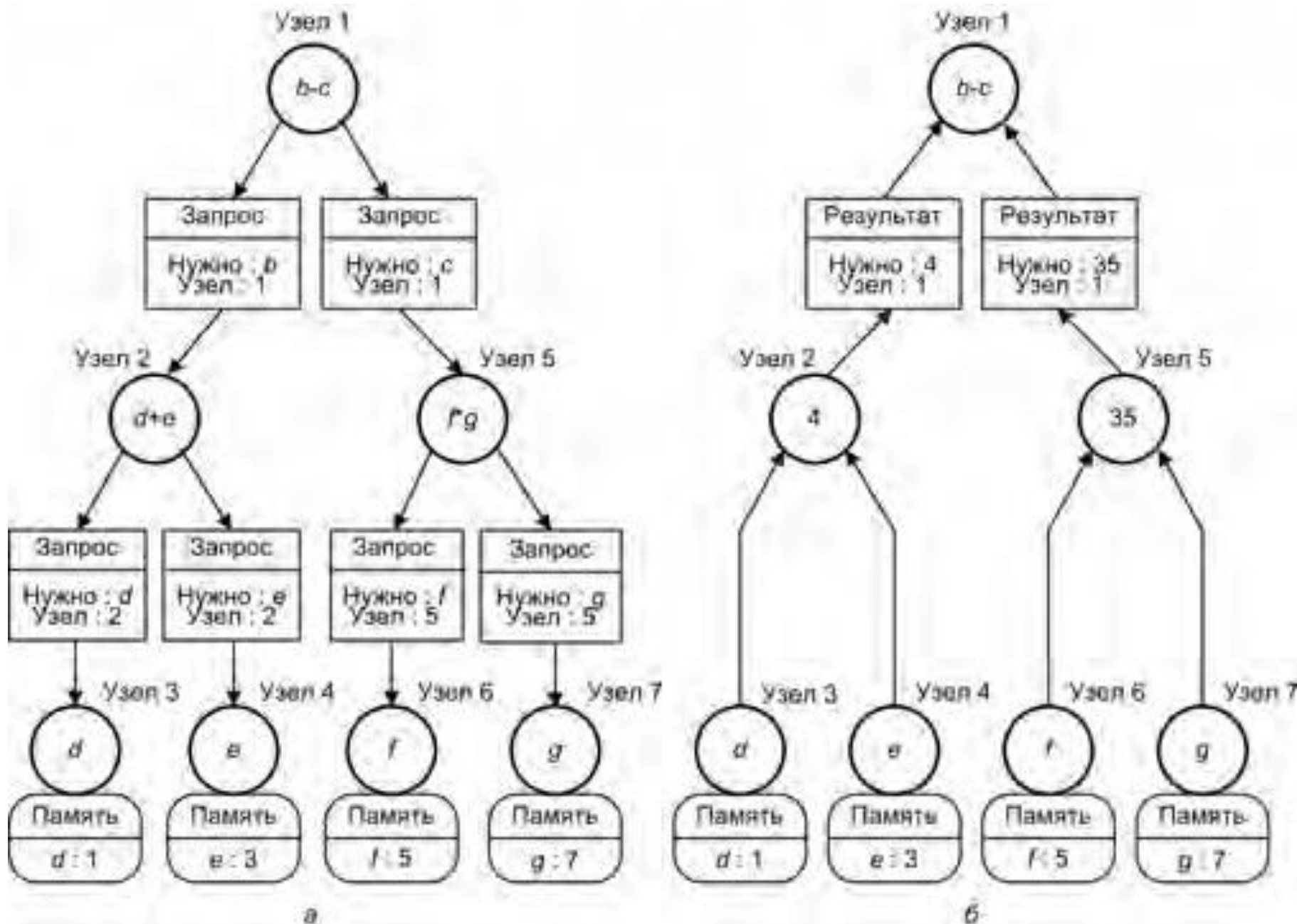


- Вершина активируется, когда на всех её входных дугах присутствуют токены с идентичным цветом
- Токен: $\langle v, \langle f, n, c, i \rangle, a \rangle$, где c определяет фрагмент кода или тело цикла, в составе реализуемой функции f , i (индекс) – цвет токена



- Любое вычисление полностью описывается *указателем команды* (IP, Instruction Pointer) и *указателем кадра* (FP, Frame Pointer)
- Токен: $\langle v, \langle FP, IP \rangle \rangle$







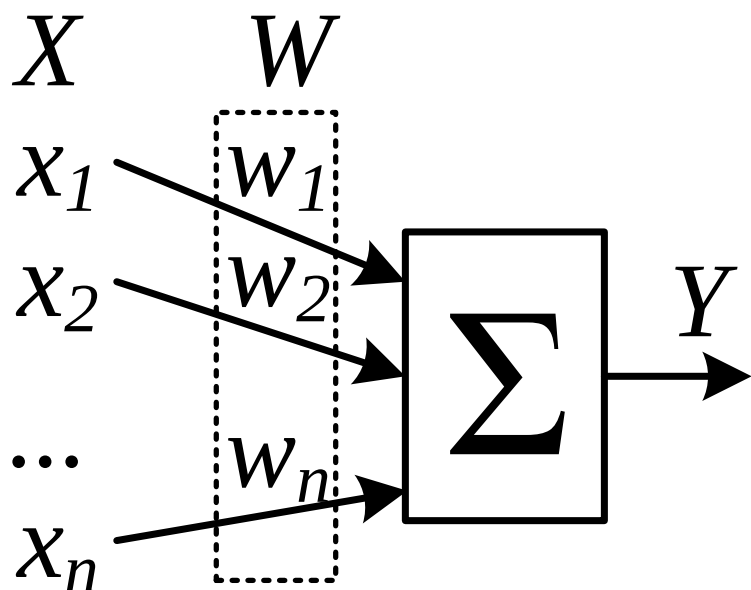
Система является когерентной, если каждая операция чтения по какому-либо адресу, выполненная любым из процессоров, возвращает значение, занесённое в ходе последней операции записи по этому адресу, вне зависимости от того, какой из процессоров производил запись последним



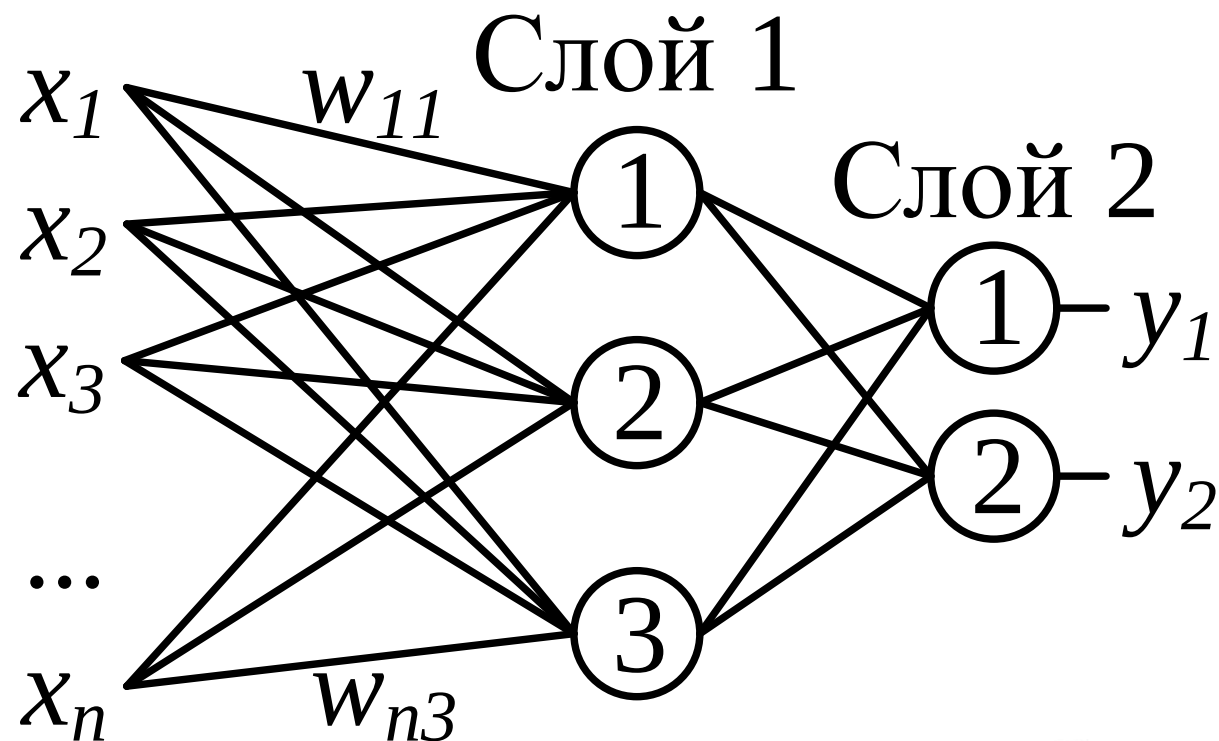
- Клеточные и ДНК-процессоры
- Нейронные процессоры
- Процессоры с многозначной (нечёткой, fuzzy) логикой
- Квантовый компьютер



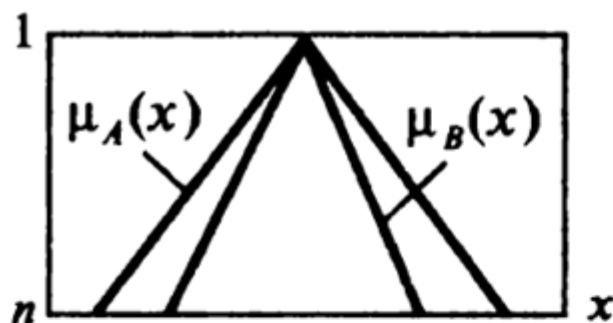
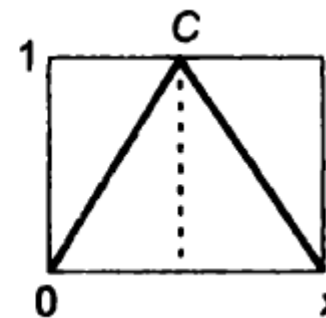
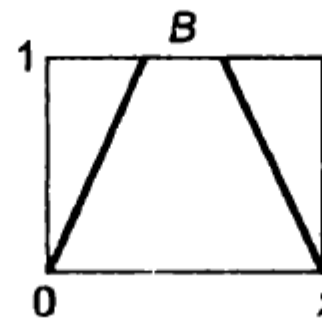
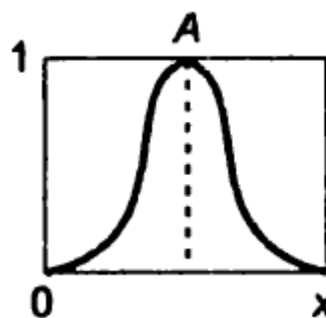
- Биокomпьютинг
- Молекулы дезоксирибонуклеиновой кислоты (ДНК) выполняют функции биопроцессора
- Структура процессора – структура молекулы ДНК
- Молекулярная и электронная составляющие – химические реакции между молекулами, поиск и выделение результата; обработка информации и анализ результатов
- Клеточные процессоры – самоорганизующиеся колонии «умных» микроорганизмов, их геном содержит некую логическую схему



$$f(x) = \frac{1}{1 + e^{-ax}}$$

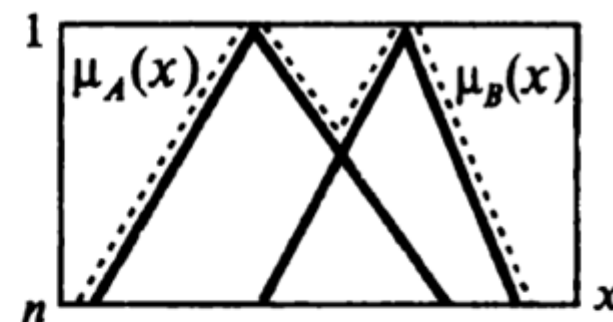


Различные типы функций принадлежности:



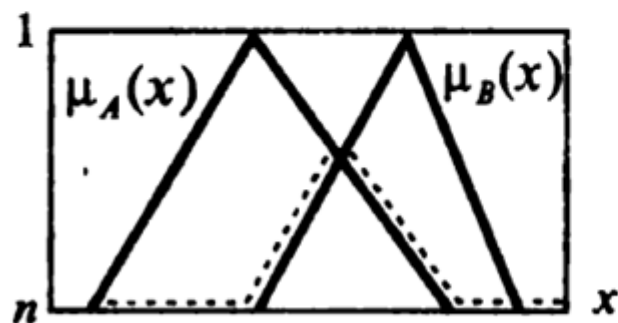
$$B \subseteq A \quad \forall x \in X$$

$$\mu_A(x) \leq \mu_B(x)$$



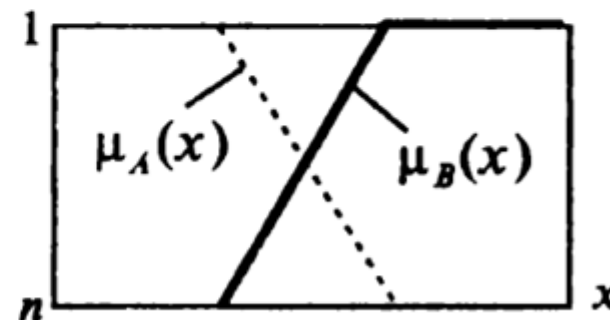
$$A \cup B \quad \forall x \in X$$

$$\mu_0(x) = \max\{\mu_A(x), \mu_B(x)\}$$



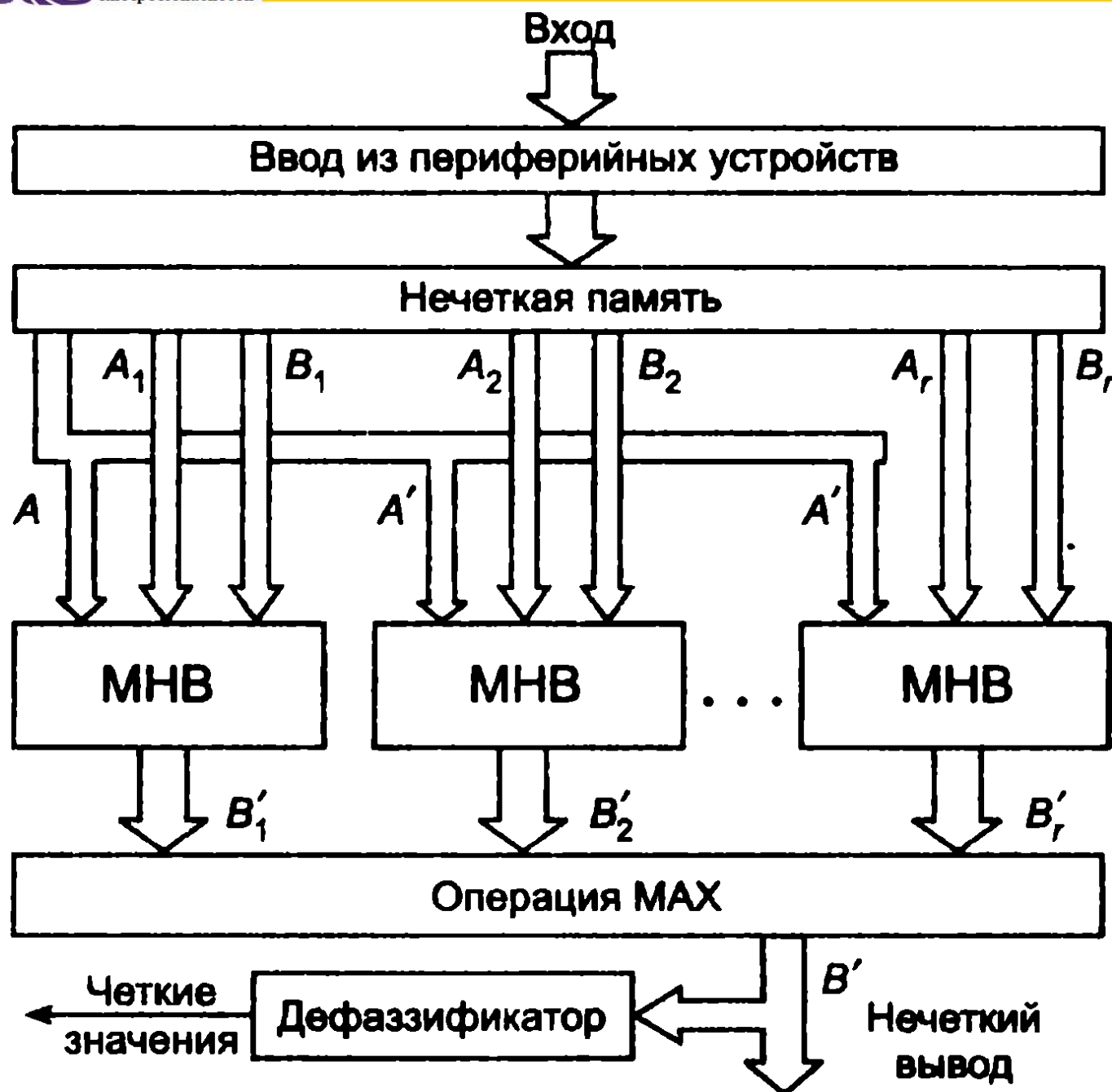
$$A \cap B \quad \forall x \in X$$

$$\mu_0(x) = \min\{\mu_A(x), \mu_B(x)\}$$



$$\forall x \in X$$

$$\mu_A(x) = 1 - \mu_B(x)$$



– МЕХАНИЗМ НЕЧЁТКОГО ВЫВОДА



- L двухуровневых квантовых элементов (кубитов) имеет 2^L линейно независимых состояний, каждое из которых может быть изменено с вероятностью $[0;1]$ – квантовое вычислительное устройство размером L кубит может выполнять параллельно 2^L операций
- Операция в квантовых вычислениях – поворот вектора состояния регистра в 2^L -мерном гильбертовом пространстве
- Вероятностный характер результата, но есть возможность приближения вероятности получения правильного результата к единице